Campus Resource Management System (CRMS)
Project Report

Project Team Members

- Sushant
- Krish Dabas
- Aryan Yadav
- Ritik Atri
- Aditya Kishore

1 Project Overview

The **Campus Resource Management System (CRMS)** is a multi-tenant Software-as-a-Service (SaaS) platform designed to streamline the allocation and scheduling of university resources such as computer labs, lecture halls, and equipment.

It replaces fragmented systems (manual logs and spreadsheets) with a centralized digital platform, ensuring:

- Efficient utilization of resources
- Elimination of double-booking
- Conflict-free scheduling

2 Technology Stack & Deployment

The system follows a decoupled client-server architecture:

Frontend (Client)

- Built with **Next.js (React)**
- Styled using **Tailwind CSS**
- Enhanced with **Framer Motion** (Midnight Neon Glassmorphism UI)
- Hosted on **Vercel**

Backend (Server)

- **Express.js API** using **TypeScript**
- Stateless RESTful architecture
- Deployed on **Render**

Database

- **PostgreSQL (Neon)**
- Managed via **Prisma ORM**

3 System Design & User Hierarchy

CRMS enforces strict multi-tenant boundaries using JWT-based authentication.

User Roles

SUPER_ADMIN (Platform Owner)

- Creates and manages institutions
- Full system-level access

ADMIN (Institution Manager)

- Operates within a specific institution
- Approves/rejects bookings
- Overrides conflicts
- Monitors operations via dashboard

STUDENT & FACULTY

- View resource availability
- Submit booking requests
- Access resource details

4 Applied Software Engineering Patterns

4.1 Strategy Pattern (Conflict Resolution)

- Location: `/patterns/strategy/`
- Enables dynamic conflict resolution strategies:
  - `StrictConflictStrategy` (First-Come-First-Serve)
  - `PriorityConflictStrategy` (Hierarchy-based override)

4.2 State Pattern (Booking Lifecycle)

- Location: `/patterns/state/`
- Enforces valid transitions:
  - `PENDING` → `APPROVED` → `CANCELLED`
- Prevents invalid state transitions

4.3 Observer Pattern (Event-Driven Architecture)

- Location: `/patterns/observer/`
- Implements Pub/Sub via `EventBus`
- Decouples core logic from notifications and logging

4.4 Factory Pattern (Object Creation)

- Location: `/patterns/factory/UserFactory.ts`
- Centralizes user object creation with role-based behavior

4.5 Mapper / DTO Pattern (Data Security)

- Location: `/mappers/`
- Prevents exposure of sensitive data
- Ensures safe API responses

5 Security & Authentication Flow

- **JWT Access Tokens**
  - Issued upon login
  - Cryptographically signed
- **Authorization Middleware**
  - `authMiddleware` for identity validation
  - `institutionGuard` for tenant isolation
- **Payload Validation**
  - Implemented using **Zod**
  - Prevents malformed inputs

6 Database Entity Relationships (Prisma)

Core Relationships

- `Institution (1) → (N) User, Resource`
- `User + Resource (1) → (N) Booking`

Data Integrity

- Soft delete using `deletedAt`
- Preserves historical data

7 Future Scaling Capabilities

- WebSocket integration (Socket.io)
- Real-time notifications and updates
- No modification required in core logic due to Observer pattern

8 SOLID Principles Implementation

8.1 Single Responsibility Principle (SRP)

- Each class has a single responsibility (e.g., BookingService, UserService)

8.2 Open-Closed Principle (OCP)

- Extendable without modifying core logic (strategies, states)

8.3 Liskov Substitution Principle (LSP)

- Subclasses safely replace base classes

8.4 Interface Segregation Principle (ISP)

- Small, focused interfaces (Strategy, State, Observer)

8.5 Dependency Inversion Principle (DIP)

- Depends on abstractions, not concrete implementations

9 Advanced Software Design Patterns

9.1 Domain-Driven Design (DDD)

- Separation into Domain, Application, and Infrastructure layers

9.2 CQRS (Partial)

- Separation of commands and queries

9.3 Repository Pattern

- Implemented via mappers

9.4 Event-Driven Architecture

- Observer + EventBus enables scalability

9.5 Multi-Tenancy Guarding

- Prevents cross-institution data leakage

10 Summary

CRMS is a production-grade system demonstrating:

- Strong use of design patterns and SOLID principles
- Secure multi-tenant architecture
- Scalable deployment (Vercel, Render, Neon)
- Clean separation of concerns

The system is engineered to:

- Prevent race conditions
- Maintain data integrity
- Scale efficiently
- Deliver a modern user experience